

fuClaw AIoT 代理人 技術教學手冊

架構、邏輯與實務深度指南
Prompt-Orchestrated Embedded Agent Edition

對話 + 推理

工具 + 視覺

記憶 + 硬體

適用平台：Realtek AMB82-mini / HUB 8735 Ultra

Build Date：2026-05-24

目 錄

第 1 章	代理人核心定義與設計哲學	3
第 2 章	系統運作邏輯鏈（Workflow Deep Dive）	6
第 3 章	軟體定義行為（SD 卡持久化配置）	10
第 4 章	多模態感官處理與編碼技術	14
第 5 章	工具箱與硬體安全規範	18
第 6 章	自主評估與連貫工作流	22
第 7 章	技術侷限與教學反思	26
第 8 章	防盜偵測技能實作 SOP	29
附錄 A	常見錯誤排查與 FAQ	33
附錄 B	延伸實作挑戰題庫	35

第 1 章 代理人核心定義與設計哲學

理解 fuClaw 的本質：從 IoT 到 AI Agent 的進化

1.1 什麼是 AI 代理人（Agent）？

傳統 IoT 裝置只能被動地「接收指令 → 執行動作」，缺乏自主判斷能力。fuClaw 的設計目標是讓嵌入式裝置成為一個能夠「感知、推理、行動」的自主實體。

系統類型	行為特徵
傳統 IoT	接收固定指令，按程式邏輯執行，無法理解自然語言或情境
fuClaw Agent	理解自然語言、視覺圖像、語音輸入，自主推理後選擇適當工具執行

核心概念	Agent = 感知（Perception）+ 推理（Reasoning）+ 行動（Action）+ 記憶（Memory）。fuClaw 在 MCU 上完整實現了這四個要素。
------	---

1.2 六大要素整合架構

fuClaw 的代理人架構由以下六個要素無縫整合而成，缺一不可：

對話	推理	工具	視覺	記憶	硬體
Telegram Bot	Google Gemini	JSON 工具路由	相機 + Gemini Vision	SD 卡持久化	GPIO 數位/類比

1.3 混合自主代理模式（Hybrid Autonomous Agent）

fuClaw 同時支援兩種運作模式，教師可引導學生理解這兩種模式的差異：

模式	說明與範例
互動模式（Reactive）	使用者透過 Telegram 傳送指令，系統即時解析並回應。如：「幫我開藍色 LED」、「拍一張照片」
自主模式（Proactive）	定時任務（periodicCheck_task）每分鐘自動觸發，執行防盜偵測等技能，無需使用者介入

教學重點	混合自主代理是 fuClaw 與一般聊天機器人的最大差異。學生應理解：代理人不只是「問答系統」，而是能主動感知環境並採取行動的實體。
------	--

1.4 提示詞驅動的工具路由（Prompt-Orchestrated Tool Routing）

fuClaw 不使用 Gemini 的原生 Function Calling API，而是採用更靈活的「提示詞驅動路由」設計：

1. 在系統提示詞中定義所有工具的 JSON 格式規範
2. Gemini 理解規範後，自行輸出符合格式的 JSON 字串
3. 本地韌體接收並解析 JSON，驗證後執行對應動作
4. 執行結果回注對話歷史，供下次推理參考

程式碼範例

```
// Gemini 輸出的 tool_call 範例
{
  "type": "tool_call",
  "method": "/digitalwrite",
  "params": {
    "pin": 24,
    "pinmode": "digitalwrite",
    "value": 1
  }
}
```

□ 設計優點

此方法讓系統在無網路原生工具支援的舊版模型上也能正常運作，且開發者對工具行為有完整控制權，安全性更高。

第 2 章 系統運作邏輯鏈

Workflow Deep Dive：從訊息接收到硬體執行

2.1 系統生命週期概覽

fuClaw 的完整運作流程分為三個階段：

階段	函式	說明
① 初始化	<code>setup()</code>	SD 卡讀取設定 → WiFi 連線 → 相機初始化 → FreeRTOS 任務建立 → 系統提示詞組建 → 對話歷史還原
② 持續輪詢	<code>task_getTelegramMessage()</code>	每 1 秒向 Telegram API 發送請求，偵測新訊息（文字或語音）
③ 防盜檢查	<code>task_anti_theft_detection()</code>	每 30 秒自動執行防盜偵測等自主技能，不依賴使用者指令
③ 排程檢查	<code>task_time_scheduling()</code>	每 60 秒自動執行防盜偵測等自主技能，不依賴使用者指令

2.2 多軌並行任務架構（FreeRTOS）

fuClaw 利用 FreeRTOS 實現多工並行，確保系統同時保持「即時回應」與「自主巡查」的能力。

FreeRTOS 任務參數說明		
任務名稱	堆疊大小	堆疊大小說明
<code>task_getTelegramMessage</code>	16384 bytes（16 KB）	處理 Telegram 長輪詢、JSON 解析、Gemini 請求，需要較大堆疊
<code>task_anti_theft_detection</code>	6144 bytes（6 KB）	僅負責定時觸發技能評估，資料量較小
<code>task_time_scheduling</code>	6144 bytes（6 KB）	僅負責定時觸發技能評估，資料量較小

教學延伸	引導學生思考：為什麼 <code>task_time_scheduling</code> 執行前需要先呼叫 <code>botClient.stop()</code> ？答：FreeRTOS 任務共用網路資源，若不停止 Telegram 連線可能造成封包衝突。
------	--

2.3 訊息路由器（handleAgentResponse）深度解析

`handleAgentResponse()` 是整個系統的「大腦分流器」，負責判斷 Gemini 的輸出並採取對應行動。其處理邏輯如下：

輸入格式	判斷結果	執行動作
以 { 開頭且以 } 結尾	單一 JSON 物件	解析 <code>method</code> 與 <code>params</code> ，呼叫 <code>executeTool()</code>
以 [開頭且以] 結尾	JSON 陣列（多步驟）	循序執行陣列中每個 <code>tool_call</code> ，遇到無效項目立即停止
"NONE" 字串	工作流程完成哨兵	不做任何動作，結束此次工作流程
其他純文字	自然語言回覆	格式化處理後透過 <code>telegramSendMessage()</code> 傳送
解析失敗	JSON 格式錯誤	記錄至 <code>Serial</code> ，通知使用者輸入 "Continue" 重試

2.4 訊息前處理流程（handleAgentResponse 預處理）

在進行 JSON 解析之前，系統必須清理 Gemini 回應中可能存在的干擾字元：

程式碼範例

```
// ① 移除多餘空白與換行
message.trim();
message.replace("\n", ""); // 清除 JSON 換行
message.replace("\t", ""); // 清除 Tab 字元

// ② 還原被跳脫的引號 (Gemini 偶爾會多加跳脫)
message.replace("\\\"", "\"");

// ③ 還原 Markdown 跳脫字元
message.replace("\\-", "-");
message.replace("\\*", "*");
```

□ 常見問題

Gemini 有時會在 JSON 前後加上 ``json 的 Markdown 程式碼區塊標記，或在文字回覆中使用 ** 加粗語法。handleAgentResponse 的純文字分支已處理這些清理邏輯。

2.5 完整請求-回應時序圖

以「使用者說：請開啟綠色 LED」為例，完整的處理流程如下：

①	使用者在 Telegram 輸入「請開啟綠色 LED」
②	getTelegramMessage() 輪詢到新訊息，取得文字內容
③	呼叫 geminiChatRequest()，將訊息連同系統提示詞送至 Gemini API
④	Gemini 解析使用者意圖，判斷需要確認後執行，回傳確認問句 (/chat)
⑤	系統透過 telegramSendMessage() 傳送確認問句給使用者
⑥	使用者回覆「確認」，再次觸發 geminiChatRequest()
⑦	Gemini 輸出 /digitalwrite JSON，pin=24, value=1
⑧	handleAgentResponse() 解析 JSON，呼叫 executeTool("/digitalwrite", params)
⑨	toolPinOutput(24, "digitalwrite", 1) 執行 digitalWrite(24, 1)
⑩	結果 JSON 存入 historicalMessages，呼叫 evaluateWorkflowContinuation()
⑪	Gemini 判斷工作流程完成，回傳 "NONE"，流程結束

第 3 章 軟體定義行為

SD 卡持久化配置：無需修改程式碼的功能擴展

fuClaw 透過 SD 卡中的五個 Markdown 檔案實現「軟體定義行為」，這是該系統最具教學價值的設計之一——讓非程式人員也能透過修改文字檔案來改變 AI 的個性、裝置定義與技能行為。

3.1 五大核心設定檔總覽

檔案名稱	格式類型	功能說明
env.json	JSON 格式憑證	存放 WiFi SSID/密碼、Telegram Token/ChatID、Gemini API Key、Time zone
soul.md	助理個性提示詞	定義 AI 的人格、語氣與決策風格，覆蓋程式碼內建的 geminiRole
device.md	硬體裝置定義	描述已確認的 GPIO 腳位對應，包含安全規則
skill.md	技能 SOP 定義	以機器可讀的 JSON 流程定義技能步驟，如防盜偵測
memory.md	對話歷史	自動儲存與還原對話上下文，確保重開機後記憶不遺失

教學設計優點	將「行為定義」與「程式碼」分離，是「配置即程式碼（Configuration as Code）」概念的實踐。學生可以在不重新燒錄韌體的情況下，修改 SD 卡檔案來實驗不同的 AI 行為。
--------	---

3.2 env.json 詳細說明

env.json 採用 JSON 格式，系統在 setup() 中透過 setEnvironmentSettings() 解析並套用。

程式碼範例

```
{
  "wifi_ssid":      "你的 WiFi 名稱",
  "wifi_pass":      "你的 WiFi 密碼",
  "telegramBot_token": "123456789:AAF...",
  "telegramBot_chatID": "987654321",
  "gemini_apikey":   "AIzaSy...",
  "timezone":       "Taiwan"
}
```

□ 安全提醒	env.json 包含重要憑證，請確保 SD 卡妥善保管。教學場合建議使用測試用的 API Key，並在課後立即撤銷。
--------	---

3.3 soul.md 助理個性設計

soul.md 的內容會直接覆蓋程式碼中的 `geminiRole` 變數，成為系統提示詞的第一層。

以下為不同教學場景的 soul.md 設計範例：

場景	soul.md 內容範例
科學實驗助理	你是一個嚴謹的科學實驗助理，回答時必須引用數據，並提醒使用者實驗安全注意事項。
親切家庭助理	你是一個溫暖友善的家庭助理，使用繁體中文回答，語氣輕鬆活潑，會適時加入鼓勵的話語。
資安偵測系統	你是一個專業的資安偵測系統，發現異常時必須立即回報，語氣簡潔專業，不做多餘解釋。
互動教學機器人	你是一位有耐心的 STEM 教師，引導學生用問答方式理解概念，每次回答後提出一個思考問題。

3.4 device.md 裝置映射設計

device.md 定義了 AI 能夠直接控制的硬體清單，這是硬體安全的第一道防線。其格式應與程式碼內建的 `devicesDefinition` 保持一致。

標準 device.md 格式範例（以 HUB 8735 Ultra 擴充外部繼電器為例）：

程式碼範例

```
=====
已確認的硬體裝置
=====

HUB 8735 Ultra (原廠)
- 綠色 LED：腳位 25
- 藍色 LED：腳位 26
- 補光 LED：腳位 13 (類比 0-255)
- 功能按鈕：腳位 12 (僅輸入)

擴充裝置 (學生自行接線)
- 客廳繼電器 (控制電燈)：腳位 4 (數位輸出)
- 溫度感測器：腳位 A2 (類比輸入)
```

3.5 skill.md 技能 SOP 設計原則

skill.md 是 fuClaw 最具創造力的設計空間。教師可以引導學生設計新技能，培養「AI 流程設計」能力。

一個良好的技能定義必須包含：

- 技能 ID 與名稱（唯一識別碼）
- 目標描述（Goal：這個技能要達成什麼）
- 機器格式步驟（MUST OUTPUT EXACT JSON ARRAY ONLY）
- 每個步驟的前置條件與後續判斷邏輯
- 執行規則（嚴格禁止輸出說明文字）

進階挑戰

引導學生設計「溫度過高警示」技能：當類比溫度感測器（`analogread`）數值超過閾值時，觸發繼電器（`digitalwrite`）並傳送警示照片（`/still`）。

3.6 memory.md 對話持久化機制

memory.md 儲存完整的 Gemini 對話歷史（historicalMessages），格式為 Gemini API 所需的 JSON messages 陣列片段。

程式碼範例

```
// memory.md 的內容範例（片段）
, { "role": "user", "parts": [{ "text": "請開啟 LED" }] }
, { "role": "model", "parts": [{ "text": "{...tool_call...}" }] }
, { "role": "user", "parts": [{ "text": "/digitalwrite" }] }
, { "role": "model", "parts": [{ "text": "{status: success}" }] }
```

□ 注意事項

隨著對話增加，memory.md 的大小會持續成長，最終導致 Gemini API 請求超出 Token 限制或裝置堆積記憶體不足。建議定期執行 `/reset` 清除對話歷史。

第 4 章 多模態感官處理與編碼技術

視覺、語音與圖像的嵌入式處理深度解析

fuClaw 整合了相機（視覺）與 Telegram 語音訊息（聽覺）兩種感知管道，讓嵌入式裝置具備超越傳統 IoT 的多模態感知能力。

4.1 視覺處理流程（geminiVisionRequest）

技術細節	說明
相機規格	預設 320×240 解析度，JPEG 格式，透過 VideoStream 函式庫擷取
圖片大小	320×240 JPEG 約 15–40 KB，Base64 後約 20–55 KB 字串
編碼方式	Base64 encoding，以每 3 bytes 為一組處理，不需要額外的上傳步驟
傳輸方式	嵌入在 JSON 的 inline_data 欄位中直接傳送至 Gemini API
frames 參數	true = 擷取新畫格；false = 使用上次快取（imageAddress/imageLength 全域變數）

程式碼範例

```
// Base64 編碼核心邏輯
char *input = (char *)fbBuf;
char output[base64_enc_len(3)];
String imageFile = "";

for (size_t i = 0; i < fbLen; i++) {
    base64_encode(output, (input++), 3); // 每次編碼 3 bytes
    if (i % 3 == 0) imageFile += String(output);
}

// 嵌入 Gemini Vision API 請求 JSON
String Data = "{\"contents\": [{\"parts\": [{\"text\": \"\" + message +
\"    \"}, {\"inline_data\": {\"mime_type\": \"image/jpeg\",
\"    \"data\": \"\" + imageFile + \"\"}]]}]}"
```

教學延伸：為什麼用 Base64？	HTTP/JSON 協定傳輸的是文字，無法直接傳送二進位圖片資料。Base64 將二進位資料轉換為可在 JSON 字串中安全傳輸的 ASCII 字元集，代價是資料量增加約 33%。
-------------------	--

4.2 /still 與 /vision 的使用時機決策樹

/still 拍照工具	/vision 視覺分析工具
使用者想看目前畫面	需要 AI 理解圖片內容
「拍一張照片給我」	「幫我分析一下現在有沒有人」
純粹記錄影像	偵測物體、人物或狀態
不進行 AI 分析	做條件式決策（如果看到...就...）
傳送至 Telegram 後完成	回傳觀察結果供後續推理使用
☐ 不可直接觸發硬體	☐ 不可直接觸發硬體（需確認）

4.3 語音處理流程（sendAudioFileToGeminiSTT）

當使用者傳送 Telegram 語音訊息時，fuClaw 會執行以下四步驟流程：

步驟	函式	說明
①	getTelegramFilePath()	從 Telegram getFile API 取得語音檔案的 CDN 路徑（OGG/Opus 格式）
②	downloadTelegramFile()	使用 HTTP/1.0 下載原始音訊位元組至堆積緩衝區（最大 256 KB）
③	sendAudioFileToGeminiSTT()	將音訊進行 Base64 編碼，以 inline_data 格式送至 Gemini 進行語音轉文字
④	geminiChatRequest()	將轉錄文字視為一般文字訊息，重新進入 Gemini 推理流程

HTTP/1.0 vs HTTP/1.1 的關鍵差異	下載語音檔案時使用 HTTP/1.0 而非 HTTP/1.1，是為了避免「分塊傳輸編碼（Chunked Transfer Encoding）」。HTTP/1.1 的分塊格式在每個資料塊前加入長度資訊，嵌入式系統需要額外解碼才能取得純二進位資料；HTTP/1.0 則直接傳送純粹的二進位本體，大幅簡化接收邏輯。
----------------------------	--

第 5 章

工具箱與硬體安全規範

確保教學實驗安全的工具設計哲學

fuClaw 的工具系統設計以「安全第一」為原則，透過多層驗證機制確保 AI 的每個硬體動作都是可控且可審計的。

5.1 完整工具清單

工具名稱	類型	說明
/digitalwrite	數位輸出	設定 GPIO 腳位為 HIGH (1) 或 LOW (0)，控制 LED、繼電器等
/analogwrite	類比輸出	PWM 輸出 0–255 數值，控制補光燈亮度、馬達速度等
/digitalread	數位輸入	讀取 GPIO 腳位狀態（0 或 1），偵測按鈕按壓等
/analogread	類比輸入	讀取 ADC 數值（0–4095），偵測溫度、光線等感測器
/syncrtc	RTC	更新硬體 RTC 實時時鐘
/getrtc	RTC	讀取硬體 RTC 當前時間
/still	靜態拍照	擷取相機畫格並直接傳送至 Telegram，不做 AI 分析
/vision	視覺分析	擷取圖片送至 Gemini Vision，回傳語意分析結果
/search	網路搜尋	啟用 Google Search 接地搜尋，取得即時資訊
/delay	延遲等待	暫停執行 0–10000 毫秒，用於 LED 閃爍、動作間隔等
/memory	記憶體診斷	回傳目前堆積記憶體用量與對話歷史長度
/log	執行歷史	顯示工具執行記錄至序列埠監視器
/reset	重設對話	清除對話歷史，回到初始系統提示詞狀態
/chat	自然語言回覆	包含回覆文字的 tool_call，傳送純文字至 Telegram
/reboot	重新啟動	觸發 NVIC_SystemReset() 重新啟動裝置

5.2 原子執行規則（Atomic Execution Rule）詳解

「原子執行」是整個安全設計的核心。每次工具呼叫必須是不可分割的最小單元：

規則	說明
一個腳位（One Pin）	每次只能操作一個 GPIO，不能「同時控制 pin 24 和 pin 23」
一個操作（One Operation）	不能在同一個 tool_call 中同時讀取和寫入
一個數值（One Value）	digitalwrite 只能是 0 或 1，不能是 "blink" 或 "toggle" 等複合指令

多步驟工作流程的正確執行方式是使用 JSON 陣列，由系統循序執行：

程式碼範例

```
// 正確：LED 閃爍一次（4 個原子操作）
[
  { "type": "tool_call", "method": "/digitalwrite",
    "params": { "pin": 24, "pinmode": "digitalwrite", "value": 1 } },
  { "type": "tool_call", "method": "/delay",
    "params": { "milliseconds": 500 } },
  { "type": "tool_call", "method": "/digitalwrite",
    "params": { "pin": 24, "pinmode": "digitalwrite", "value": 0 } },
  { "type": "tool_call", "method": "/delay",
    "params": { "milliseconds": 500 } }
]
```

5.3 多步驟 JSON 陣列的最長有效前綴規則

當陣列中的某個 tool_call 不完整或無效時，系統不是「跳過無效項目」，而是「立即中止，執行到此為止」。這稱為「最長有效前綴（Longest Valid Prefix）」規則。

範例	若 Gemini 輸出包含 3 個 tool_call，其中第 2 個缺少必要參數，則系統只執行第 1 個，第 2 和第 3 個全部捨棄。這防止了在不完整計畫中執行錯誤動作。
----	---

5.4 數值驗證機制

工具	有效數值範圍	邊界行為
digitalwrite	0 或 1（整數）	其他數值回傳 invalid_digital_value 錯誤，不執行
analogwrite	0–255（整數）	constrain() 強制限制在有效範圍內，不會溢出
delay	0–10000（毫秒）	constrain() 強制限制，防止無限期卡住主流程
digitalread	無需輸入數值	純讀取，回傳 0 或 1
analogread	無需輸入數值	純讀取，回傳 ADC 原始值

5.5 確認機制（Global Device Control Policy）

為防止 AI 在未獲授權的情況下自行修改硬體狀態，系統設有強制確認機制：

10. 所有直接由使用者請求的硬體動作，執行前必須獲得明確確認
11. 由技能自動觸發的硬體動作（如防盜偵測中的 LED 閃爍）免除確認
12. 若使用者要求「以後不用確認」，系統必須再次要求確認此政策變更

教學討論

為什麼需要確認機制？引導學生思考：如果 AI 誤解指令，在沒有確認的情況下直接控制工業設備，可能造成什麼後果？這是工業 IoT 中「命令確認」設計模式的教學入口。

第 6 章 自主評估與連貫 workflow

OODA 循環與 evaluateWorkflowContinuation 深度解析

fuClaw 與一般「問一答一」聊天機器人的最大差異，在於其能夠自主評估工作流程狀態，並在不需要使用者持續介入的情況下完成多步驟任務。

6.1 OODA 循環模型

fuClaw 的自主決策模型借鑑了軍事決策理論中的 OODA 循環（Observe-Orient-Decide-Act）：

循環階段	對應元件	說明
Observe（觀察）	/vision、/digitalread	透過視覺或 GPIO 感測環境目前狀態
Orient（定向）	Gemini 推理	分析感測結果，結合使用者任務與歷史記憶，判斷情境
Decide（決策）	tool_call JSON 輸出	Gemini 決定下一步動作，以 JSON 格式輸出
Act（行動）	executeTool()	執行 GPIO 控制、搜尋、拍照等具體動作

6.2 evaluateWorkflowContinuation() 詳解

這個函式是 OODA 循環的「Re-Observe」機制，在每個工具執行後自動詢問 Gemini：「任務完成了嗎？」

程式碼範例

```
void evaluateWorkflowContinuation(bool reCheck, String task = "") {
  if (!reCheck) return; // 最後一個工具才評估

  String prompt =
    "Analyze the execution result and determine whether the workflow is complete.\n"
    + "If additional hardware actions are required, return ONLY tool_call JSON.\n"
    + "If workflow is complete, return EXACTLY: NONE."

  handleAgentResponse(geminiChatRequest(prompt, 1));
}
```

6.3 reCheck 旗標設計邏輯

在 JSON 陣列執行中，每個 executeTool() 呼叫的 reCheck 參數設計有特別的邏輯：

tool_call 位置	reCheck 行為
中間的 tool_call（非最後一個）	reCheck = false，不觸發評估，直接執行下一個工具
最後一個 tool_call	reCheck = true，執行完畢後觸發 evaluateWorkflowContinuation()

設計原因	若每個工具執行後都觸發評估，在 LED 閃爍（開→等待→關→等待）的場景中，每次都會中斷流程向 Gemini 詢問，造成大量多餘的 API 呼叫。只在最後一步評估可大幅降低 token 消耗。
------	--

6.4 "NONE" 哨兵機制

當 Gemini 判斷工作流程完成時，必須精確回傳字串 "NONE"（全大寫）。系統在 `handleAgentResponse()` 中明確檢查此值並終止流程，不做任何其他動作。

❏ 常見陷阱

若 Gemini 回傳 "None"、"none" 或 "工作流程完成" 等變體，系統不會識別為哨兵，而是將其作為自然語言傳送至 Telegram。在 `soul.md` 和 `skill.md` 中應明確強調哨兵格式的嚴格要求。

6.5 任務鏈（Task Chaining）範例

以「搜尋今日天氣，若高雄下雨則關閉補光燈」為例，展示完整的自主任務鏈：

#	執行動作	Gemini 推理過程
1	<code>/search</code> 「高雄今日天氣」	執行 Google Search，獲取天氣資料
2	<code>evaluateWorkflowContinuation()</code>	搜尋結果：雨天。詢問使用者確認是否關閉補光燈
3	使用者確認	收到確認後，Gemini 決定執行 <code>/analogwrite pin=13, value=0</code>
4	<code>/analogwrite pin=13, value=0</code>	補光燈亮度設為 0（關閉）
5	<code>evaluateWorkflowContinuation()</code>	Gemini 判斷任務完成，回傳 NONE

第 7 章 技術侷限與教學反思

認識嵌入式 AI 的邊界，培養工程師思維

優秀的工程師不僅要知道系統「能做什麼」，更要深刻理解「不能做什麼」以及「為什麼」。本章引導教師帶領學生正視 fuClaw 的技術限制。

7.1 記憶體管理挑戰

項目	說明
問題來源	Arduino String 類型在 C++ 中每次修改都可能觸發記憶體重新配置（malloc/free），長時間運行後造成堆積碎片化（Heap Fragmentation）
影響症狀	系統運行一段時間後，malloc() 返回 NULL（記憶體不足），可能導致語音緩衝區無法配置、Base64 編碼失敗
監控方法	使用 /memory 命令查看 xPortGetFreeHeapSize()（當前空餘）與 xPortGetMinimumEverFreeHeapSize()（歷史最低），趨近 0 時需要 /reset
緩解策略	定期執行 /reset、減少 geminiMaxOutputTokens、縮短 soul.md 長度、在關鍵操作前呼叫 free() 釋放快取

教學延伸：堆積碎片化模擬	可以用一張白紙代表記憶體，學生用不同大小的便利貼代表每次 malloc 的記憶體塊，free 後留下空洞。展示為何最終即使「總空餘空間足夠」，也可能無法配置一個連續的大塊記憶體。
--------------	---

7.2 CPU 運算開銷

操作	CPU 影響分析
Base64 編碼	一張 320×240 JPEG（約 20 KB）需要處理約 27,000 次迭代，期間 FreeRTOS 其他任務會被短暫延遲
Gemini API 請求	包含圖片的請求內容可達 50–100 KB，以 1024 bytes 為單位分塊傳送，佔用大量序列傳輸時間
視覺任務成本	視覺任務（/vision）的總耗時包含：擷取 + Base64 編碼 + HTTPS 連線 + API 處理 + 回應解析，通常需要 5–15 秒

7.3 網路穩定性

問題	說明與建議
Gemini API 逾時	預設 20 秒逾時，教學環境的共享 WiFi 可能頻繁觸發逾時，導致「Gemini did not respond」錯誤
Telegram 長輪詢	若 WiFi 斷線，Telegram 任務會進入 WiFi 重連循環，此期間無法接收任何指令
SSL 握手延遲	每次與 Telegram 或 Gemini 建立新的 SSL 連線都需要完成 TLS 握手，在訊號弱的環境可能耗時 2–5 秒

教學準備建議	進行課堂展示前，建議確認：① 專用 WiFi AP 頻道不擁擠 ② 測試 Gemini API Key 的配額是否充足 ③ 提前執行 /reset 清除過長的對話歷史。
--------	--

7.4 Gemini 輸出的不確定性

問題類型	說明與影響
溫度 (Temperature) 影響	geminiTemperature = 1.0 代表較高的輸出隨機性。在精確工具呼叫場景，偶爾可能輸出略有差異的 JSON 格式，被 ArduinoJson 拒絕
JSON 格式汙染	Gemini 有時在 JSON 前後加入說明文字或 Markdown 標記，handleAgentResponse 的前處理雖有清理，但複雜情況下仍可能解析失敗
NONE 哨兵變體	Gemini 可能輸出 "None"、"DONE"、"完成" 等語意相近但格式不符的字串，導致工作流程無法正確終止

7.5 工程反思問題（課堂討論題）

13. 如果 geminiMaxOutputTokens 設為 500，一個複雜的 LED 閃爍 JSON 陣列可能被截斷，會發生什麼？
14. 為什麼即使 Wi-Fi 訊號良好，geminiVisionRequest() 有時也會回傳 "Connection failed"？
15. 如果兩個 FreeRTOS 任務同時嘗試寫入 memory.md，會發生什麼問題？目前程式碼是否有保護？
16. 如何在不修改程式碼的情況下，讓系統每 5 分鐘（而非 60 秒）才進行一次防盜偵測？

第 8 章 防盜偵測技能實作 SOP

完整的多步驟自主技能設計範例

本章以內建的 `anti_theft_detection` 技能為完整範例，帶領教師逐步拆解技能設計的每個細節，並提供擴充挑戰。

8.1 技能目標與觸發方式

屬性	值
技能 ID	anti_theft_detection
目標	偵測相機畫面中是否有人類，若有則傳送警示圖片並閃爍 LED 三次
觸發方式	由 <code>periodicCheck_task</code> 每 60 秒自動觸發（自主模式）
豁免確認	是（此技能由系統自動執行，不需使用者確認硬體動作）
適用裝置	HUB 8735 Ultra（使用 pin 26 藍色 LED）

8.2 技能執行流程逐步解析

步驟	工具呼叫	說明與決策邏輯
Step 0	<code>evaluateWorkflowContinuation()</code>	<code>periodicCheck_task</code> 注入提示詞，要求執行 <code>anti_theft_detection</code> 技能
Step 1	<code>/vision (frames: true)</code>	擷取新畫格，查詢「圖中是否有人類」，等待 Gemini Vision 分析
判斷 A	（無人）→ <code>NONE</code>	Gemini Vision 回傳無人，任務完成，回傳 <code>NONE</code> 結束此次循環
判斷 B	（有人）→ 繼續	Gemini Vision 確認有人，繼續執行警示工作流程
Step 2a	<code>/still (frames: false)</code>	傳送先前快取的圖片（已偵測到人的那張）至 Telegram，不重新擷取
Step 2b	<code>/digitalwrite pin=26, value=1</code>	開啟藍色 LED
Step 2c	<code>/delay 500ms</code>	等待 0.5 秒
Step 2d	<code>/digitalwrite pin=26, value=0</code>	關閉藍色 LED
Step 2e	<code>/delay 500ms</code>	等待 0.5 秒（完成一次閃爍，可重複 2b-2e 共三次）
Final	<code>evaluateWorkflowContinuation()</code>	Gemini 確認三次閃爍完成，回傳 <code>NONE</code> 結束技能

8.3 為什麼 `/still` 使用 `frames: false` ？

關鍵設計決策	<code>/vision</code> 已在 Step 1 擷取並快取了圖片（ <code>imageAddress/imageLength</code> 全域變數已更新）。 <code>/still</code> 使用 <code>frames: false</code> 是為了傳送「Gemini 當時分析到有人的那張圖片」，而非此刻又重新擷取的新畫面（人可能已離開）。這確保了警示圖片與偵測時間點的一致性。
--------	---

8.4 完整的 skill.md 防盜技能定義

程式碼範例

```
=====
BUILT-IN SKILLS REGISTRY
=====

Skill ID: anti_theft_detection
Name: 防盜偵測
Goal: 偵測人員存在並觸發警示工作流程

SKILL EXECUTION (MACHINE FORMAT)
MUST OUTPUT EXACT JSON ARRAY ONLY

Step 1: /vision 偵測
{
  "type": "tool_call", "method": "/vision",
  "params": { "query": "是否有人類?", "frames": true,
    "task": "有人繼續；無人返回 NONE" }
}

Step 2: 警示序列（有人時輸出此陣列）
[
  {"/still", frames:false}, {"/digitalwrite", pin:26, value:1},
  {"/delay", 500ms},          {"/digitalwrite", pin:26, value:0},
  {"/delay", 500ms} // 重複三次...
]
```

8.5 技能擴充挑戰

以防盜偵測為基礎，引導學生進行以下進階擴充：

難度	挑戰內容
初級擴充	修改 skill.md，讓 LED 閃爍從 3 次改為 5 次，並在閃爍完成後發送一條文字訊息「警告：偵測到入侵者！」
中級擴充	新增「白天防盜」邏輯：先用 /analogread 讀取光線感測器，若亮度足夠（白天）才進行 /vision 偵測，節省夜間誤報
高級擴充	設計「溫度異常警示」新技能：每 5 分鐘讀取溫度感測器，超過 35°C 時觸發補光燈全亮（模擬警示燈）並傳送通知
創意擴充	設計「來客問候」技能：偵測到人臉後，透過 /chat 傳送個性化的問候語，並拍照上傳作為來訪紀錄

第 附錄 A 章 常見錯誤排查與 FAQ

常見錯誤訊息對照表

錯誤訊息	原因	解決方案
JSON parse failed	Gemini 輸出格式不符	輸入「Continue」重試；若頻繁發生，考慮調低 geminiTemperature
Connection failed	WiFi 或網路問題	檢查 WiFi 連線；確認 API Key 有效；檢查防火牆設定
Previous image does not exist	使用 frames:false 但無快取圖片	先執行 /still 或 /vision (frames:true) 建立快取
Malloc failed for Base64 encoding	堆積記憶體不足	立即執行 /reset 清除對話歷史；用 /memory 監控剩餘空間
Gemini did not respond	Gemini API 20 秒逾時	檢查網路品質；確認 API 配額；考慮減少圖片解析度
Invalid digital value	digitalwrite 數值不是 0 或 1	檢查 Gemini 輸出的 value 欄位；確認提示詞中的驗證規則

常見教學問題解答

Q：為什麼修改了 soul.md，但 AI 的行為沒有改變？

A	修改 SD 卡檔案後必須重新啟動裝置（/reboot 或重新上電），系統才會重新讀取 soul.md 並更新 geminiRole 變數。
----------	---

Q：/log 沒有在 Telegram 顯示工具歷史，只說「請查看序列埠監視器」？

A	這是設計決定。工具執行歷史（executeToolHistory）可能很長，超過 Telegram 單訊息字數限制（4096 字元）。序列埠監視器可以顯示完整記錄。如需在 Telegram 查看，可修改程式碼只傳送最後 N 條記錄。
----------	--

Q：如何在不重新燒錄韌體的情況下新增新的硬體裝置？

A	只需修改 SD 卡中的 device.md，加入新裝置的名稱和 GPIO 腳位，並重新啟動裝置。系統在 setup() 時會讀取 device.md 並覆蓋預設的 devicesDefinition 變數。
----------	---

第 附錄 B 章 延伸實作挑戰題庫

基礎挑戰（建議第 1–2 週）

17. 在 Telegram 傳送「讓 LED 閃爍 5 次後傳送一張照片給我」，觀察系統如何分解並執行多步驟任務。
18. 修改 `soul.md`，讓 AI 以「科學家」身份回答問題，並測試語氣是否有改變。
19. 在 `device.md` 中新增一個「假想裝置」（如：空調，腳位 5），然後要求 AI 開啟空調，觀察安全機制如何阻止執行。

進階挑戰（建議第 3–4 週）

20. 設計一個「天氣感知補光」技能：搜尋天氣，晴天則降低補光燈亮度，陰天則提升至最大亮度。
21. 修改 `periodicCheck_task` 的等待時間，從 60 秒改為 30 秒，並觀察對系統穩定性的影響。
22. 實作一個「語音控制 LED」的完整流程：用語音說「開燈」，驗證語音轉文字→意圖推理→GPIO 控制的完整鏈路。

專題挑戰（建議期末專案）

23. 設計並實作一個「教室環境監測系統」：整合溫度感測器（`analogread`）、光線感測器（`analogread`）和相機（`vision`），每小時自動生成一份環境報告並傳送至 Telegram。
24. 擴充 `handleAgentResponse()`，讓系統能在解析失敗時自動重試一次，並記錄失敗原因至 `executeToolHistory`。
25. 設計「記憶體自動管理」機制：在 `historicalMessages` 長度超過 50,000 字元時，自動截斷最舊的對話紀錄，確保系統長時間穩定運行。